Open Community Radio Starter Kit

A Professional Workflow Automation for Independent Stations

by Helen Bolek

Setup & Documentation Manual

October 28, 2026

Contents

1 Beyond Radio: A Universal Booking System **4**

2 Motivation: Empowering Cultural Exchange **5**

3 Why Google Workspace? (And How to Get It for Free) **6**
3.1 Free for Nonprofits 6
3.2 Everything in One Place 6

4 Community Retention: The Mailing List and SOS List **8**
4.1 The Shared Mailing List (Newsletter) 8
4.2 The SOS List (Spontaneous Coverage) 8

5 Visual Management: The Color-Coding System **9**
5.1 Individual Show Slots 9
5.2 Broadcast Days & Studio Hosts 9
5.3 Blocked Days & Special Events 9

6 The Four Core Modules **11**
6.1 Module 1: The Application System 11
6.2 Module 2: DJ Schedule Checker 13
6.3 Module 3: SOS Alert System 15
6.4 Module 4: Automated Supporting Membership & SEPA Workflow 16

7 Day-to-Day Operations (Handling Exceptions) **18**
7.1 Handling Cancellations (Making a slot available again) 18
7.2 Finalizing Supporting Members 18
7.3 Lost Notification Emails 18

8 Technical Quirks & Google Workspace Secrets **19**
8.1 The “Sent Folder” Mystery in Shared Drives 19
8.2 The Google Groups Email Trick (Routing to the Inbox) 19
8.3 Sending as an Alias 19
8.4 The Timezone Trap (Why your shows are scheduled wrong) 19
8.5 Google’s Daily Email Limits & Script Quotas 20
8.6 The “Bus Factor” and Shared Drives 20
8.7 Backups & Safety Net 20

9 Locating Your Setup IDs **21**
9.1 Finding a Spreadsheet ID 21
9.2 Finding a File ID (Statutes & Privacy Policy PDFs) 21
9.3 Finding the Calendar ID 21

10 Before You Begin: The Prerequisites **22**

11 Setting Up the Code: What Goes Where? **23**
11.1 Setup for Modules 1 & 3: Application System and SOS Alerts 23
11.2 Setup for Module 2: DJ Schedule Checker 24
11.3 Setup for Module 4: Supporting Membership & SEPA Workflow 24
11.4 Filling In the Configuration 24

12 Going Live, Authorization & Testing **26**
12.1 Testing & Playing Around (The Sandbox) . . . 26
12.2 Deploying the Web App (First Time Only) . . . 26
12.3 Updating Later (Keep Your Links!) . . . 26
12.4 The Scary (But Harmless) Google Security Warning . . . 27
12.5 The Executions Log (How to panic correctly) . . . 27

13 Data Privacy & Security (GDPR/DSGVO) **28**
13.1 Security Built Into the Code . . . 28
13.2 Securing the Master Spreadsheets . . . 28
13.3 Privacy Information at the Point of Collection . . . 28
13.4 Routine Data Purges . . . 28
13.5 Data Processing Agreement with Google . . . 28
13.6 Data Subject Rights (Access & Deletion) . . . 29

14 Implement to your website (iFrame Integration) **30**
14.1 The Embedding Code . . . 30
14.2 Tips for a Seamless Integration . . . 30

15 License & Getting Help **31**

1 Beyond Radio: A Universal Booking System

While this manual uses an independent radio station as its primary example, the core problem this toolkit solves is universal. Whether you are running a nail salon, a tattoo studio, a shared rehearsal space, or a consultation service, managing booking requests scattered across Instagram DMs, chaotic email threads, and missed phone calls quickly becomes a logistical nightmare.

This toolkit provides a centralized, fully automated solution to that chaos. By simply renaming the text fields in the HTML forms and adjusting the calendar slots to fit your specific operating hours, you can easily adapt this entire Google Workspace ecosystem to fit any project. It is, at its heart, a highly adaptable organizational tool for anyone who needs reliable scheduling, automated email confirmations, and a clean, collaborative database without paying for expensive subscription software.

Before you start, here's a quick overview of what you'll need:

Checklist

- A Google account (ideally **Google Workspace for Nonprofits** for higher sending and storage limits)
- A GitHub account to download or copy the code
- Admin access to your station's Google account (to set up aliases and sharing permissions)
- Basic willingness to copy and paste code – no programming knowledge required

2 Motivation: Empowering Cultural Exchange

Community radio stations are much more than just broadcasting hubs; they are vital spaces for cultural exchange, subculture representation, and local community building. They give a voice to grassroots artists, minority groups, and passionate music enthusiasts who often do not find a platform in mainstream media.

However, running a community radio requires significant organizational effort. Unfortunately, the drive to create and connect is frequently hindered by a lack of technical know-how. Coordinating dozens of volunteers, managing show schedules, and communicating efficiently can quickly become overwhelming.

This toolkit is designed to eliminate these technical barriers. By automating the most tedious administrative tasks, I want to empower people with zero coding experience to run a professional, smoothly operating radio station.

No project starts flawlessly, and ours was no exception. When my friends and I founded our community radio station a year and a half ago, we faced a steep learning curve. Every single day, we encountered workflows that were either inefficient, overly tedious, or simply didn't scale. After eighteen months of managing everything manually from scratch and learning the hard way how easy it is to lose track of emails, shows, and people, I finally mapped out exactly which processes needed optimization. Rome wasn't built in a day, and neither is a thriving community radio station. It takes time, trial, and error to truly understand your bottlenecks and build the systems that set you free.

3 Why Google Workspace? (And How to Get It for Free)

A Note on Big Tech & Pragmatism

It is no secret that relying on a massive corporate entity like Google is not the ideological ideal for an independent, grassroots community radio station. In a perfect world, our entire digital infrastructure would run on self-hosted, open-source platforms.

However, after extensive evaluation of various alternatives, we had to face reality: for a volunteer-run project with limited technical resources, setting up, securing, and maintaining custom servers is a massive barrier to entry. Building a station is exhausting enough. Google Workspace simply works out of the box. Ultimately, we chose pragmatism over ideological perfection. This ecosystem is currently the easiest, most reliable, and most accessible way to get a station off the ground, allowing the team to immediately focus on what actually matters: the music, the culture, and the community.

When building a digital infrastructure for a radio station, accessibility and cost-efficiency are the top priorities. We chose Google Workspace because it provides a powerful, all-in-one ecosystem that eliminates the need for expensive third-party tools or scattered local files.

3.1 Free for Nonprofits

The biggest advantage for community projects: Google offers **Google Workspace for Nonprofits** completely for free to eligible registered charities and associations (e.V. in Germany). The free tier includes:

- **Custom Email Addresses:** Professional addresses at your station's domain (e.g., `hello@your-radio.com`).
- **User Accounts:** Up to 2000 free user accounts for your team members.
- **Storage:** 30 GB of cloud storage per user across Google Drive, Gmail, and Google Photos.
- **Shared Drives:** Centralized team folders where files belong to the organization, not individual users.

3.2 Everything in One Place

By relying on Google Workspace, you avoid the classic "radio chaos" of having audio files on one person's hard drive, the schedule in a paid external tool, and the application forms in messy Word documents.

Instead, this toolkit utilizes the synergy of built-in Google tools:

- **HTML Web Apps (Apps Script):** Serve as the public-facing application portals.
- **Google Sheets:** Act as the central, clean database for all DJs, supporting members, the SOS List and the Mailing List.
- **Google Calendar:** Automatically acts as the visual schedule and data source for the website.
- **Gmail:** Handles all automated background communications and BCC mailings.

Because it is entirely cloud-based, your whole team can access the same clean, organized data from anywhere in the world, directly from their browser, without installing any software.

Running an independent or community radio station usually involves a lot of manual administrative work: managing show applications, tracking down DJs, handling spontaneous cancellations, and coordinating schedules.

This **Open Community Radio Starter Kit** replaces messy email threads, scattered chat groups, and manual calendar entries with a fully automated Google Workspace ecosystem that was built with data protection (GDPR/DSGVO) in mind.

What changed? Previously, station managers had to manually copy applicant data from forms to spreadsheets, create calendar events by hand, type out confirmation emails, and use private messenger apps for urgent slot fillings. With this toolkit, the entire process is automated. Data flows seamlessly from a web form into a structured database, the calendar updates itself, and communication is handled via official station channels with zero manual data entry.

4 Community Retention: The Mailing List and SOS List

Before detailing the technical modules that automate the station, it is crucial to understand two core concepts this toolkit uses to actively manage and maintain the community: the Mailing List and the SOS List. Both lists are filled automatically whenever someone applies for a show or a supporting membership, eliminating the need to manually collect contact data.

4.1 The Shared Mailing List (Newsletter)

A radio station thrives on its community, but keeping everyone informed can be difficult. The **Mailing List** is an opt-in newsletter list that lives in its own Google Sheet. Instead of manually passing around a sign-up sheet at events or trying to reach people via social media algorithms, both public forms – the show application and the supporting membership application – ask people if they want to stay in the loop. This list becomes your most valuable tool for announcing station news, summer breaks, upcoming festivals, or invitations to open assemblies. It turns one-time applicants into a lasting community.

Every sign-up is stored in the same format:

Email	Name	DJ Alias	Source	Added on
max@example.com	Max Example	DJ Example	Show Application	27.10.2026 12:00

- **No duplicates:** If an address is already on the list, it is not added again. A missing name or DJ alias is simply filled in.
- **Proof of consent:** The "Added on" column records when someone signed up – useful if you ever need to show where an address came from.
- **Regular DJs:** Residents who never filled out a form can be added by hand with the source "Regular Show".
- **Unsubscribing:** Simply delete the row.

4.2 The SOS List (Spontaneous Coverage)

Empty airtime is the biggest fear of any live radio manager. Shows get canceled last minute due to illness, travel delays, or personal emergencies. Traditionally, managers scramble to find a replacement by texting people individually, making stressful phone calls, or spamming chaotic, unmoderated WhatsApp groups.

The **SOS List** solves this recurring problem. It is a separate, dedicated opt-in pool of DJs who explicitly state during their application that they are willing and able to jump in on short notice (1 to 7 days). This creates a respectful, automated standby roster that allows the station to seamlessly fill sudden gaps in the programming.

5 Visual Management: The Color-Coding System

A busy community radio calendar can quickly turn into an unreadable wall of text. To prevent this, we established a strict, highly visual color-coding system in Google Calendar. This allows the entire team to understand the station's status for the week at a single glance.

5.1 Individual Show Slots

Regular broadcast slots are coded by their booking status. This integrates perfectly with our automated application script:

- **Blue (Free):** These are open, available slots. Their title must contain the word "Slot" (e.g., "Slot 1"). The application script scans for these to offer them to applying DJs.
- **Yellow (Requested):** As soon as a DJ submits an application through the web form, the script replaces the free slot with a yellow event. This signals to the team: *Pending review.*
- **Green (Confirmed):** Once the management clicks "Accept" on the application email, the script automatically turns the event green, meaning the slot is officially locked in.

Good to know: Yellow and green events are never offered as free slots, even if their title happens to contain "slot" – so a DJ called "Slotmachine" won't accidentally reopen their own show.

5.2 Broadcast Days & Studio Hosts

To keep track of who is actually managing the studio and broadcasting stream on a given day, we use **All-Day Events** labeled as "Sendetag" or "Broadcast Day". We use the description field of these events to write down which team members are hosting/engineering the shift.

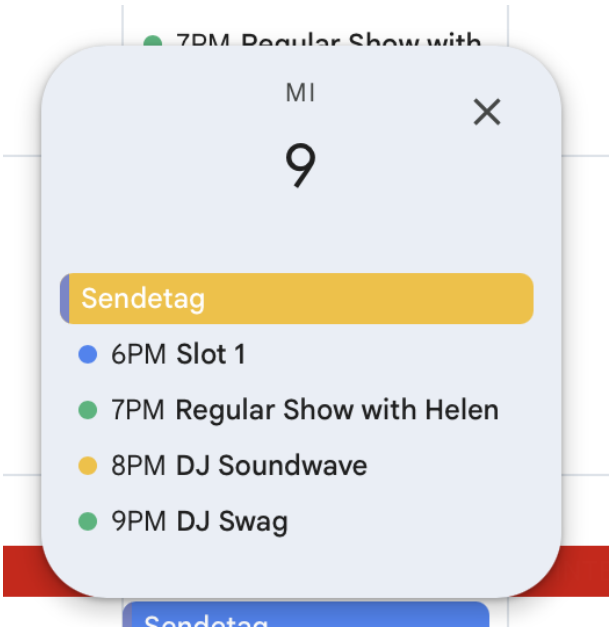
- **Blue:** Not started planning yet
- **Yellow:** Pending/waiting for information (show titles, confirmations etc.). The script sets this automatically as soon as one show of that day is pending.
- **Lavender (Pending Hosts):** All shows of the day are confirmed, but we still need team members to sign up for studio duty. The script sets this automatically.
- **Green (Broadcast Ready):** All host shifts are covered. The day is fully staffed and ready to go. This color is set by hand – the script never changes a green broadcast day.

5.3 Blocked Days & Special Events

Not every day is a regular broadcasting day. We use specific All-Day Events to block out the calendar for periods where we have no influence on the regular programming.

- **Red (Blocked):** Used for summer/winter breaks, studio maintenance, or special external events (like a festival takeover, where somebody else curates the programme).

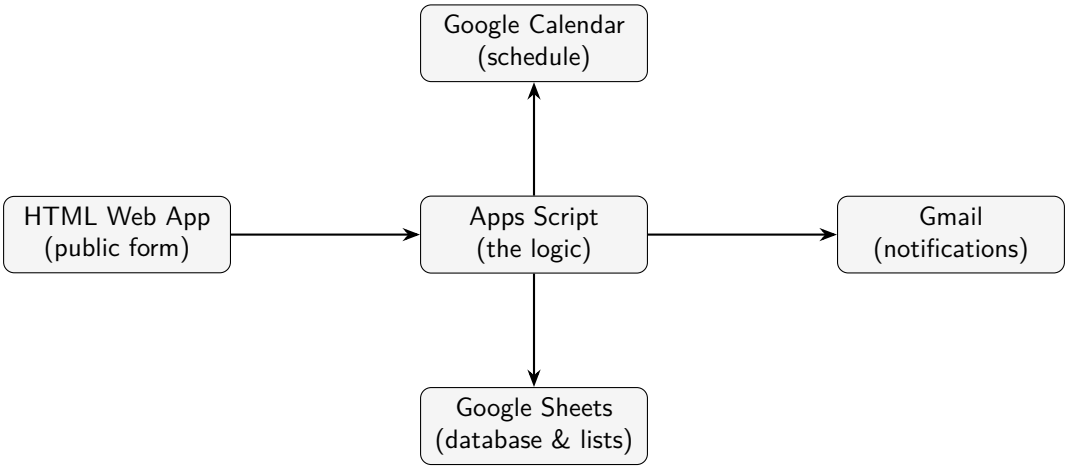
The automated advantage: Our application script is smart enough to read the calendar. If it detects a blocking All-Day Event on a specific date, it will automatically hide all slots on that day from the public application form. You never have to worry about DJs accidentally booking a show during your winter break!



SO	MO	DI	MI	DO	FR	SA
29	30	1. Dez.	2 Sendetag 6PM DJ FUN 7PM Regular Show with 2 weitere	3	4	5 Sendetag 4PM Regular Show with 5PM DJ Love 6PM DJ no swag
6	7	8	9 Sendetag 6PM Slot 1 7PM Regular Show with 2 weitere	10	11 WINTER BREAK	12 Sendetag 4PM Regular Show with 2 weitere
13 WINTER BREAK	14	15	16 Sendetag 6PM Slot 1 3 weitere	17	18	19 Sendetag 4PM Regular Show with 2 weitere
20 WINTER BREAK	21	22	23 Sendetag 6PM Slot 1 3 weitere	24	25	26 Sendetag 4PM Regular Show with 5PM DJ Love 6PM DJ no swag

6 The Four Core Modules

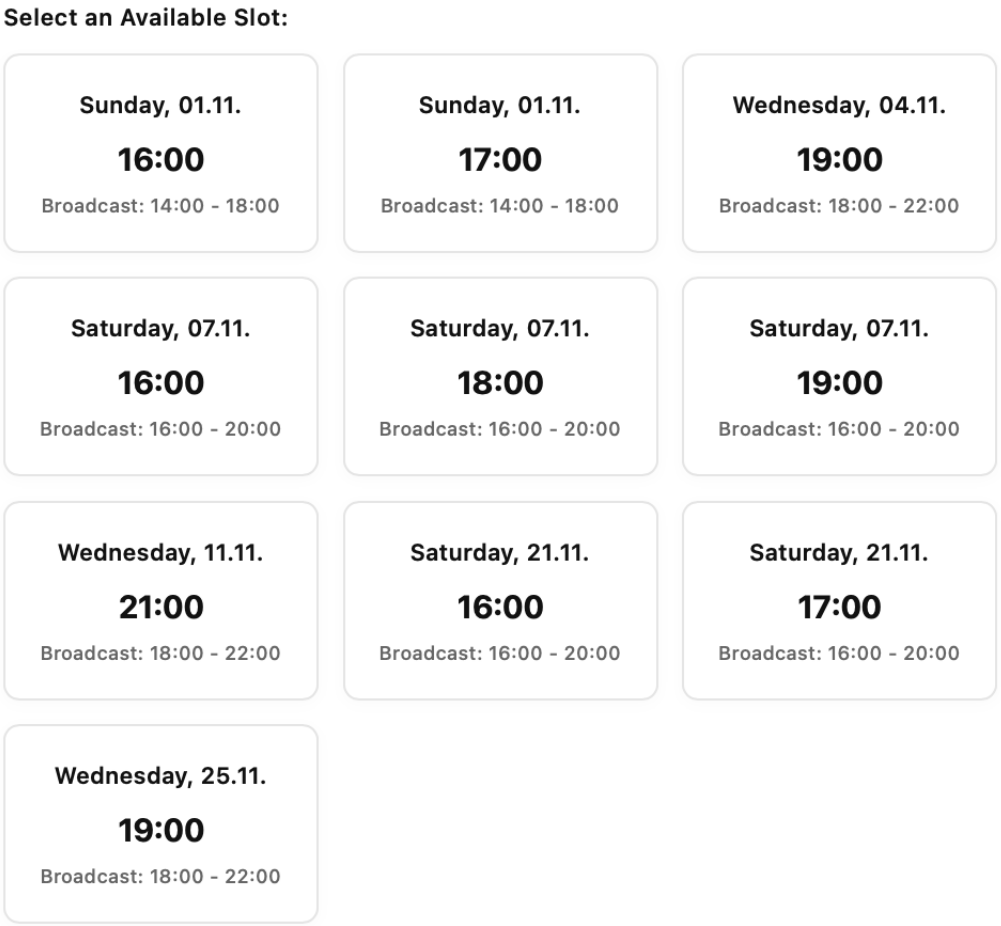
Before diving into the individual modules, here's the rough data flow of the whole system:



6.1 Module 1: The Application System

Purpose: A public-facing web application that allows DJs to apply for open broadcast slots, automatically sorting them into the Schedule, the Mailing List, and the SOS List.
How it works:

1. The script scans the station's Google Calendar for available slots (ignoring blocked days).



2. The DJ submits the form, providing all essential details upfront to eliminate tedious back-and-forth emails before the show. A well-structured application form should ask for:

   - **Basic Info & Contacts:** Real Name, DJ Alias, and Email Address.
   - **Phone Number:** Crucial for day-of emergencies (e.g., if the DJ is running late, the door is locked, or there are sudden technical issues).
   - **Show Concept:** A brief description of the musical genre or talk topic.
   - **Technical Setup & Needs:** A clear statement in the form outlining what gear the studio already provides (e.g., standard club setup) and a text field for the DJ's specific requirements (e.g., bringing a live synth, needing extra cables, or requiring technical support). This ensures smooth handovers.
   - **Agreements & Opt-ins:** Mandatory agreement to the station's Code of Conduct, plus the optional checkboxes for the Mailing List and SOS List.

3. **First come, first served:** Before booking, the script checks once more that the slot is still free and handles only one application at a time. Two people can never book the same slot – if somebody was faster, the applicant sees a friendly message and the list of free slots reloads.

4. The script replaces the free slot with a *pending* (yellow) calendar event containing all show details. The event takes over the length of the original slot.

5. The script logs the DJ's data into the `Applications` tab of the main Google Sheet (status: *Pending*).

6. The script sends a notification email to the studio management.

   Email Template: Notification to Station Management

   Subject: New Application: [DJ Alias] for [Date & Slot]

   A new request has been submitted for **[Date & Slot]**. The slot is currently blocked as yellow (pending) in your calendar.

   Name: [Real Name] ([DJ Alias])
   Email: [Email Address]
   Phone: [Phone Number]
   Concept: [Submitted Concept Text]

   [Quick Action Buttons for Accept & Decline]

7. **Decision Time (Accept vs. Decline):** The notification email includes two 1-click buttons.

   If the management clicks **"Accept"**, the script executes immediately in the background: It changes the calendar event to *confirmed* (green), sets the status in the sheet to *Confirmed*, and automatically emails the DJ the good news. Clicking the button a second time does nothing, so nobody receives the confirmation twice.

Email Template: Automatic Confirmation sent to DJ

Subject: Your Resonance Radio Show: [Date & Slot]

Hi [DJ Name],

Awesome news! We would love to have you play on [Date & Slot].

Your slot is confirmed. Please let us know if you have any questions before the show.

Cheers,
Resonance Radio Team

However, if the management clicks **"Decline"**, the system behaves differently *by design*. Instead of sending a cold, automated rejection, the button opens a pre-filled draft in your email client. This leaves room for the management to add personal feedback, explain why the slot cannot be offered, or suggest alternative dates. This ensures transparency and maintains a welcoming community spirit, rather than rejecting applicants out of egoism. (Afterwards, you have to manually turn the yellow event back into a free slot – see Section 7.1.)

Email Template: Pre-filled Draft for Decline (Allows Personalization)

Subject: Your Resonance Radio Application - [Date & Slot]

Hi [DJ Name],

Thank you so much for applying for [Date & Slot]. Unfortunately, we cannot offer you this specific slot *[... space to add personal feedback or alternate dates ...]*
We'd love to see you apply for another date soon!

Best,
Resonance Radio Team

6.2 Module 2: DJ Schedule Checker

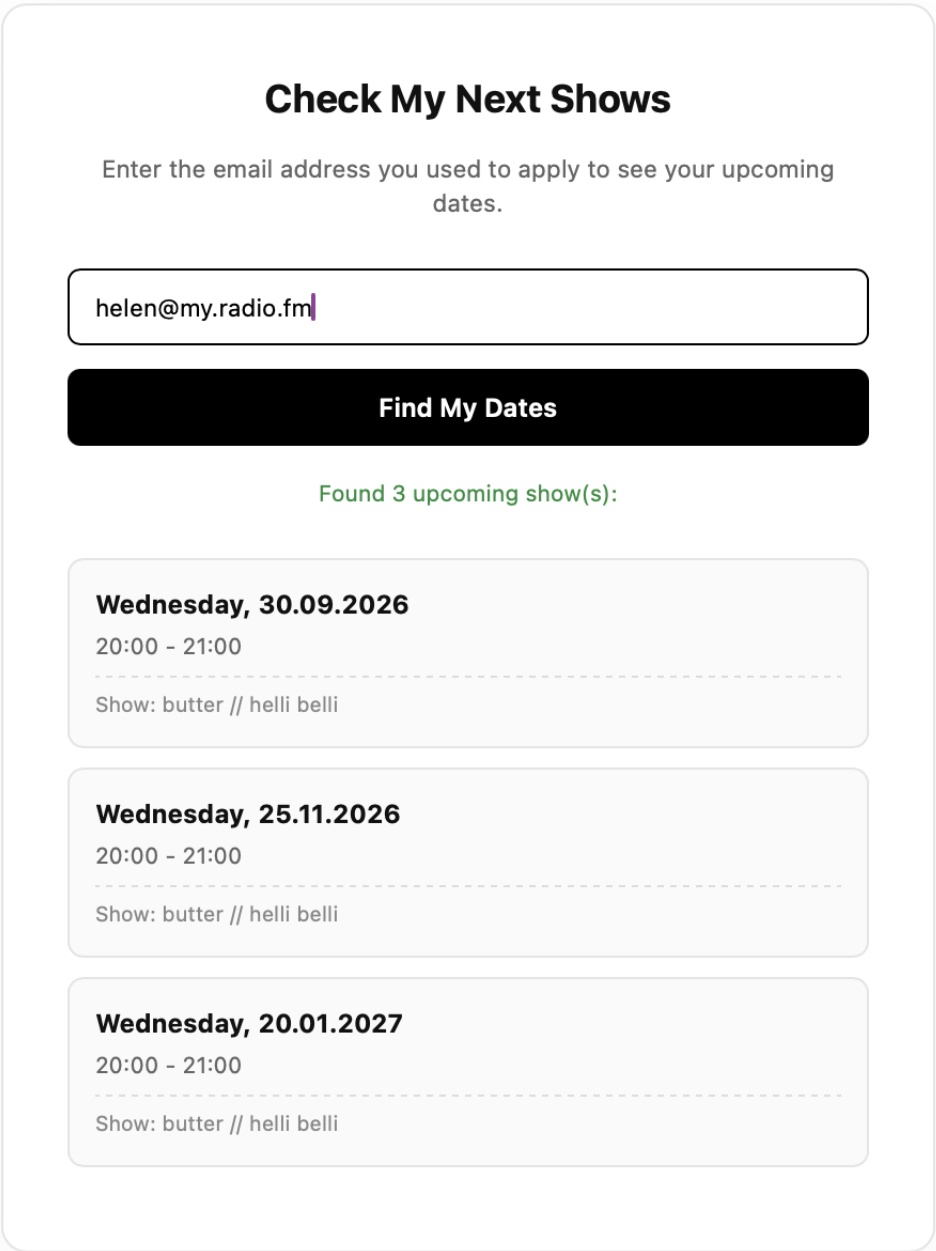
Purpose: A self-service tool for one time or regular and returning DJs to check their upcoming broadcast dates independently.

The Problem: Resident DJs who host a show regularly (e.g., every 4 to 8 weeks) often struggle to keep track of their dates or add them to their private calendars. This led to a constant stream of messages to the station management asking, "When is my next show again?" or requests from DJs wanting a full list of all their dates for the rest of the year. Looking up and typing out these schedules by hand was an incredibly tedious and time-consuming chore for the team.

How it works: When an applicant uses the booking tool (Module 1), the system automatically injects all submitted form data – including their email address – directly into the description of the Google Calendar event. This creates a fully searchable backend. For regular shows you create by hand, simply write the DJ's email address somewhere into the event description.

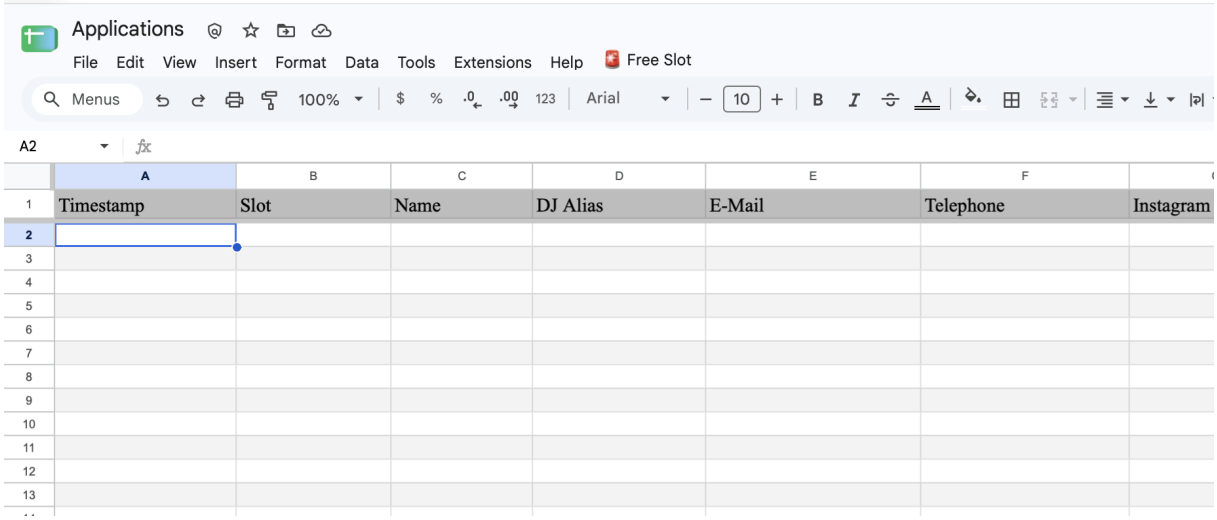
To prevent scraping of station data by third parties, the Schedule Checker acts as a secure lookup gate. Instead of publishing a public roster, the DJ simply enters the email address they used to apply. The script then scans the descriptions of all future calendar events for this *complete* email address. If a match is found, it automatically retrieves, sorts, and displays all upcoming dates for that specific person.

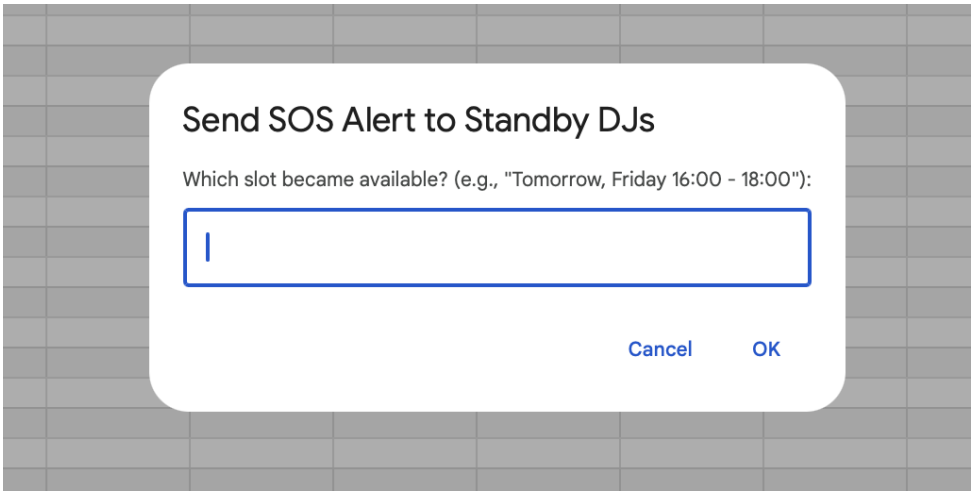
Privacy first: Only someone who knows a complete email address can see the dates linked to it. Partial addresses or whole domains (e.g., `@gmail.com`) return nothing.



6.3 Module 3: SOS Alert System

Purpose: A seamless, spreadsheet-integrated tool to utilize the SOS List and fill spontaneous slot cancellations without relying on messenger apps.
How it works: Station managers have a custom menu called "🚨 Free Slot" directly inside their Applications spreadsheet, right next to "Help". By clicking "🚨 Send SOS Alert to DJs", managers simply type the date of the dropped slot. The script automatically gathers all opted-in emails from the `SOS List` tab, removes duplicates, and sends one Blind Carbon Copy (BCC) email via the official station address. First DJ to reply gets the slot. This does not happen automatically, since the SOS Tool is not linked to the Calendar. You have to add the Person that spontaneously jumps in to your Calendar manually the same way you have to remove the person which is missing their slot.





Email Template: Automated BCC SOS Alert sent to DJs

Subject: 🚨 Resonance Radio SOS: Spontaneous Open Slot!

Hey Resonance Radio DJ,

A slot just became available on short notice and we are looking for someone to jump in!

Slot: [Input from Management, e.g., Tomorrow, 16:00 - 18:00]

If you have time and want to play, just reply directly to this email. First come, first served!

Cheers,
Your Resonance Radio Team

6.4 Module 4: Automated Supporting Membership & SEPA Workflow

Purpose: A digital application workflow for supporting members that keeps the board in control, minimizes administrative overhead and prepares a pre-filled SEPA direct debit mandate.
How it works:

1. The prospective supporting member fills out the public-facing web form, providing personal/company details, contribution amounts, payment intervals, and SEPA bank information. They also agree to the Code of Conduct via an interactive pop-up. The script checks all entries before saving them – including the IBAN checksum, so typos are caught immediately.

2. **The Veto Phase (Board Review):** To protect the association from unwanted or misaligned entities, the data is initially logged into the `Members` tab with a status of *Pending Veto* (highlighted in yellow), and a notification email is automatically dispatched to the board.

   Email Template: Notification to Station Management (Veto Phase)

   Subject: New Application: [First Name] [Last Name]

   Applicant: [First Name] [Last Name] (or company name)
   Type: [Member Type]
   Motivation: [Submitted Motivation Text]

   The data has been added to the sheet. If you accept this person/company, click Accept. Only then will the PDF be generated and sent for signature.

   [Quick Action Buttons for Accept & Veto]

3. **Decision Time (Accept vs. Veto):**

   If the board clicks **"Accept & Send PDF"**, the script triggers automatically in the background: It compiles a precise, LaTeX-style HTML-to-PDF document pre-filled with the applicant's data, updates the spreadsheet status to *PDF sent (Waiting for signature)*, and emails the applicant. The button only works while the status is *Pending Veto* – a second click never sends the PDF twice.

Email Template: Automatic PDF & Policy Package sent to Applicant

Subject: Action Required: Sign your Resonance Radio Membership

Hi [First Name],

Fantastic news! We have reviewed your application and are thrilled to welcome you as a supporting member of Resonance Radio e.V.

To complete the process and activate your SEPA mandate, we just need your signature.

Next steps:

(a) Attached to this email, you will find our statutes and privacy policy (DSGVO). Please read through them.

(b) Open the attached, pre-filled membership application.

(c) Manually or digitally check the missing boxes for the statutes, DSGVO and the SEPA direct debit mandate on the application.

(d) Sign the document (signature of the account holder) and simply reply to this email with the signed PDF.

As soon as we receive the PDF, you are officially on board!

Best,
Resonance Radio Team

However, if the board chooses to issue a veto, clicking the **"Veto"** button opens a pre-filled draft in your email client via a `mailto:` link, allowing management to handle the rejection respectfully and personally.

4. **Final Legal Handshake:** Once the applicant reviews the attached statutes and privacy policy, ticks the remaining checkboxes on the PDF (statutes, privacy policy and SEPA mandate), signs it, and emails it back, the station management **manually** sets the status to *Active* (the cell turns green automatically).

SEPA Direct Debits: What Your Association Needs

To collect membership fees via SEPA direct debit, your association needs:

- a **creditor identifier** (*Gläubiger-Identifikationsnummer*). In Germany you apply for it online at the Deutsche Bundesbank – it is free.
- an agreement with your own bank that allows the association to submit direct debits.

Enter the creditor identifier and the association's postal address in the `CREDITOR_ID` and `CREDITOR_ADDRESS` settings of Module 4. The PDF then contains a complete mandate block: creditor name, address and identifier, the type of payment (*recurring*) and a note that the **mandate reference** will be communicated separately. As long as one of the two settings is empty, the line on the PDF stays blank and the Executions log shows a warning.

Before the first debit, you must send the new member the mandate reference together with a **pre-notification**: the amount and the collection dates, normally at least 14 calendar days in advance. For recurring fees of the same amount, one notice before the first debit is enough. The mandate must be signed by the account holder. Please have your board (or your bank) check your mandate text and process; this manual is not legal advice.

7 Day-to-Day Operations (Handling Exceptions)

The automated system is designed to handle the heavy lifting, but human oversight is still required for edge cases and final approvals.

7.1 Handling Cancellations (Making a slot available again)

The automated system relies entirely on Google Calendar as its source of truth. If a DJ cancels their confirmed show (or you declined an application), you do not need to edit any code or mess with the spreadsheet. Simply open your Master Calendar and turn the event back into a free slot:

1. Rename the event so that its title contains "Slot" again (e.g., "Slot 2").
2. Set its color back to the calendar's default color (blue). Yellow and green events are never offered, even if they are called "Slot".

The application script will instantly start offering it to new applicants on the website. Alternatively, delete the event and create a fresh "Slot" event at the same time.

7.2 Finalizing Supporting Members

The SEPA workflow pauses at the "PDF sent" stage to ensure the applicant physically or digitally signs the mandate. Once the signed document arrives in your inbox, a board member opens the `Members` tab, enters the mandate reference and sets the status to *Active*. Then the member must receive the mandate reference together with the pre-notification (amount and collection dates) before the first debit ideally via email (this email must be sent manually!). This manual step is your official audit trail for the finance department. The `Calculations` tab only counts members with the status *Active*.

7.3 Lost Notification Emails

If a notification email to the board ever gets lost, nothing is lost in the sheet. Open the Apps Script editor of Module 4, select the function `resendPendingNotifications` at the top and click **Run**. Every application that is still *Pending Veto* is sent to the board again, with a working Accept button.

8 Technical Quirks & Google Workspace Secrets

Building automation within Google Workspace is fantastic, but Google's ecosystem has a few hidden rules, limitations, and architectural behaviors that can catch beginners completely off guard. This section explains the common "gotchas" you might encounter and how to avoid them before they cause confusion.

8.1 The "Sent Folder" Mystery in Shared Drives

When you build these tools, you will likely store the Google Sheet and the attached Apps Script in a Shared Drive for your whole team. However, Google Apps Script inherently executes under the personal Google Account of the person who originally authorized and deployed it, rather than a generic organization account.

What goes wrong: If you (as the script creator) deploy the application, and the script automatically emails a DJ to confirm their show, the rest of your team will not find this email in the shared team inbox's "Sent" folder. Instead, it will sit exclusively in the "Sent" folder of *your own personal Gmail account.*

Why it happens: Google treats the script as if *you* personally clicked send behind the scenes, even though the email is stamped with the station's address. Knowing this immediately saves the team from panicking or thinking the email never went out!

8.2 The Google Groups Email Trick (Routing to the Inbox)

Community radios rarely pay for expensive individual inboxes for every department. Instead, they use a shared email address (like `show@resonance-radio.fm`) configured as a *Google Group* or an *Alias* that distributes incoming emails to multiple team members.

What goes wrong: If your Apps Script tries to send an automated notification *from* the group alias directly *to* that same group alias (for example, notifying the team of a new DJ application), Google's internal spam architecture will often silently drop the email. It detects a system sending a message to itself and assumes it is a routing loop or spam.

The Solution: In the code provided in this toolkit, you will notice that the automated notification emails actively CC the script owner using `Session.getEffectiveUser().getEmail()`. By dragging a real user account into the CC line, you force Google's routing protocol to recognize the email as legitimate, ensuring it reliably lands in the team's shared inbox without getting swallowed by spam filters.

8.3 Sending as an Alias

To ensure automated emails do not reveal your personal email address to applicants, the script uses the `from: SENDER_ALIAS` parameter.

Crucial Setup Step: For this command to work, the specific Google Account that deployed the script *must* have the station's shared email address configured as an official alias. You must log into the deploying account, go to **Gmail Settings > Accounts > Send mail as**, and add your station's shared address (e.g., `show@resonance-radio.fm`). If this is missing, Google will either ignore the command and send from your private address, or the script will crash entirely with an authorization error.

8.4 The Timezone Trap (Why your shows are scheduled wrong)

Google Calendar, Google Sheets, and Google Apps Script each maintain their own independent timezone settings. If your Apps Script project defaults to Pacific Time while your calendar is set to Central European Time, the automated booking script will calculate and schedule all DJ slots multiple hours late!

The Solution: Before publishing your script for the first time, open the Apps Script Editor, click on the **Project Settings** icon (the gear on the left sidebar), and verify that the **Time zone** matches your local station's timezone exactly.

8.5 Google's Daily Email Limits & Script Quotas

Google places strict, invisible caps on how much background work a script can do to prevent abuse. These limits become critical as your station grows:

- **Daily Email Caps:** Standard free Gmail accounts can send up to 500 emails per day, while official Google Workspace accounts can send up to 2000.
- **The SOS List Trap:** If your station builds an SOS list with 600 subscribers and you try to trigger a BCC email to all of them at once from a free consumer account, the script will crash mid-execution and Google will lock your inbox for 24 hours. Keep your SOS list pruned to active, reliable standby DJs, or use a Google Workspace for Nonprofits account.
- **Execution Time Limit:** A single script run can last at most 6 minutes before Google forcefully kills it. While normal operations take seconds, massive loops or heavy data processing can hit this ceiling.

8.6 The "Bus Factor" and Shared Drives

When a core team member inevitably steps back or leaves for an Erasmus semester abroad, the station's digital infrastructure must not break.

The Rule: Never build these tools, forms, or spreadsheets inside your private "My Drive". Always create a dedicated **Shared Drive** for station management. Files in a Shared Drive belong to the organization, not the individual creator. If the original author eventually deletes their Google account, the scripts, forms, and databases inside a Shared Drive will continue to function seamlessly for whoever takes over.

8.7 Backups & Safety Net

Before making major changes to a script or restructuring a spreadsheet, always create a safety net. Google Sheets automatically keeps a version history (**File > Version history > See version history**) which allows you to roll back mistakes. However, because code depends heavily on exact tab names and column positions, don't rely on it blindly. Before carrying out structural updates, make a manual safety copy via **File > Make a copy**.

9 Locating Your Setup IDs

To connect the code to your specific station, you need to replace the placeholder values in the configuration block at the top of each `Code.js` file.

9.1 Finding a Spreadsheet ID

Open your Google Sheet. Look at the URL in your browser. The ID is the long string of random characters located between `/d/` and `/edit`. You need this for the Applications spreadsheet, the Supporting Members spreadsheet and the Mailing List.

```
https://docs.google.com/spreadsheets/d/YOUR_SHEET_ID_HERE/edit
```

9.2 Finding a File ID (Statutes & Privacy Policy PDFs)

Module 4 attaches your statutes and your privacy policy to the welcome email. Upload both as PDFs to your Shared Drive, open them, and copy the ID from the URL – it sits between `/d/` and `/view`.

```
https://drive.google.com/file/d/YOUR_FILE_ID_HERE/view
```

9.3 Finding the Calendar ID

1. Open Google Calendar on a computer.
2. In the left sidebar, hover over your station's calendar, click the three dots, and select **Settings and sharing**.
3. Scroll down to the **Integrate calendar** section.
4. Look for **Calendar ID**. It usually looks like a long email address (e.g., `example@group.calendar.google.com`).

10 Before You Begin: The Prerequisites

Before you copy any code, you must establish the foundation of your new automated system. Do not skip these steps, as the scripts rely on exact naming conventions and permissions to function correctly.

1. **The Master Calendar:**
   Do not use a personal calendar. Create a fresh, dedicated Google Calendar specifically for your broadcast schedule (e.g., "Resonance Radio Broadcasts"). This keeps things clean and prevents the script from accidentally reading your private appointments.

2. **The Database Templates (Google Sheets):**
   The code looks for exact tab names and column positions. To save you the hassle of building this by hand and risking typos that will break the code, we have prepared three perfectly formatted templates:

   - **Applications** – tabs `Applications` and `SOS List` (Modules 1 & 3)
   - **Supporting Members** – tabs `Members` and `Calculations` (Module 4)
   - **Mailing List** – tab `Mailing List` (shared by Modules 1 & 4)

   Each template also has an `About` tab that explains its columns and shows an example row.

   - Click here to open the official templates: **[LINK]**
   - For each of the three files, go to **File > Make a copy** and save the copy to your station's Shared Drive. You will use these copies for all the next steps.
   - Don't rename the tabs or move columns – the scripts write into exactly these positions.

3. **Statutes & Privacy Policy:**
   Upload your statutes and privacy policy as PDFs to the Shared Drive (needed for Module 4).

4. **The Email Alias:**
   Ensure the Google Workspace account you are currently logged into has the authority to send emails on behalf of the station. Go to **Gmail Settings > Accounts > Send mail as** and verify that your station's shared email address is listed and verified.

11 Setting Up the Code: What Goes Where?

Google Apps Script acts as the engine behind this toolkit. You don't need to know how to write code to use it, you just need to know how to copy and paste the files from GitHub into your Google Workspace.

The toolkit consists of three Apps Script projects: Modules 1 and 3 share one project (attached to the Applications spreadsheet), Module 2 runs as an independent web application, and Module 4 has its own project (attached to the Supporting Members spreadsheet).

Here's how the repository is organized:

```
Open-Community-Radio-Starter-Kit/
|-- 1_Application_System_and_SOS/
|   |-- Code.js
|   `-- Index.html
|-- 2_DJ_Schedule_Checker/
|   |-- Code.js
|   `-- Index.html
|-- 3_Supporting_Membership/
|   |-- Code.js
|   |-- Index.html
|   `-- PdfTemplate.html
|-- LICENSE
`-- README.md
```

A note on file names: On GitHub, the script files end in `.js` so that the code is displayed nicely. In the Apps Script editor, the same file is called `Code.gs` – simply paste the content of `Code.js` into `Code.gs`.

11.1 Setup for Modules 1 & 3: Application System and SOS Alerts

This project needs to be attached to your Applications spreadsheet so it can log incoming applications and add the SOS menu.

1. Open your copy of the **Applications** spreadsheet.
2. In the top menu, click on **Extensions** and select **Apps Script**. This opens the editor in a new tab.
3. You will see a default file called `Code.gs`. Delete the placeholder code inside it, and paste the entire text from the `Code.js` file found in the **1_Application_System_and_SOS** folder on GitHub.
4. Hover over the "Files" section on the left sidebar, click the + (Add a file) button, and select **HTML**. Name it exactly `Index` (with a capital I).
5. Delete the default HTML code that appears, and paste the code from GitHub's `Index.html` into this new file.
6. Fill in the configuration block (see Section 11.4) and click the floppy disk icon at the top (or press Cmd/Ctrl + S) to save.
7. Reload your spreadsheet: the "🚨 Free Slot" menu (Module 3) now appears next to "Help". The SOS code is already part of `Code.js` – there is no separate file to add.

11.2 Setup for Module 2: DJ Schedule Checker

Because this tool acts as an independent search gate and needs its own public web address, we will create a standalone script for it. This prevents its web interface from colliding with the Application System.

1. Go to your Google Drive (`drive.google.com`).
2. Click the + **New** button on the left, go to **More**, and select **Google Apps Script**. This creates a fresh, independent project. *(If you don't see it, click "Connect more apps" and search for Apps Script).*
3. Click on the default `Code.gs` file, delete the placeholder, and paste the text from the `Code.js` file inside the **2__DJ__Schedule__Checker** GitHub folder.
4. Click the + button under "Files", select **HTML**, and name it exactly `Index`.
5. Paste the text from the corresponding `Index.html` file on GitHub.
6. Fill in the configuration block and save the project. *(Remember to deploy this one as its own separate Web App following the Go-Live instructions below!)*

11.3 Setup for Module 4: Supporting Membership & SEPA Workflow

Because this module handles data logging, vetoes, and PDF generation, its code and templates need to be added to the Supporting Members spreadsheet.

1. Open your copy of the **Supporting Members** spreadsheet (the one with the `Members` tab).
2. In the top menu, click on **Extensions** and select **Apps Script**. This opens the editor in a new tab.
3. Delete the placeholder code in `Code.gs` and paste the entire text from `Code.js` in the **3__Supporting__Membership** folder.
4. Hover over the "Files" section on the left sidebar, click the + (Add a file) button, and select **HTML**. Name it exactly `Index` (with a capital I) and paste the content of `Index.html`.
5. Click the + button again under "Files", select **HTML**, and name it exactly `PdfTemplate`. Paste the content of `PdfTemplate.html` – it creates the clean, academic print layout of the SEPA application.
6. Fill in the configuration block and click the floppy disk icon at the top (or press Cmd/Ctrl + S) to save the entire project.

11.4 Filling In the Configuration

Every `Code.js` starts with a clearly marked configuration block. Replace every placeholder with your own values – everything else can stay as it is.

Setting	What to enter
Modules 1 & 3 (Applications project)	
`STATION_NAME`	Your station's name – used in all emails
`CALENDAR_ID`	ID of your broadcast calendar
`SPREADSHEET_ID`	ID of your Applications spreadsheet
`MAILING_LIST_SHEET_ID`	ID of your Mailing List spreadsheet
`ADMIN_EMAIL`	Address that receives new applications
`SENDER_ALIAS`	Station address used as sender (must be a "Send mail as" alias)
`DAYS_IN_FUTURE`	How many days ahead DJs can book (default: 60)
`BROADCAST_HOURS`	Broadcast times shown under each slot – adapt or remove
Module 2 (Schedule Checker)	
`STATION_NAME`	Same value as above
`CALENDAR_ID`	Same calendar ID as above
Module 4 (Supporting Members project)	
`STATION_NAME`	Your station's name
`LEGAL_NAME`	Official name of your association (e.g., "Resonance Radio e.V.") – printed on the PDF and in the SEPA mandate
`SHEET_ID`	ID of your Supporting Members spreadsheet
`ADMIN_EMAIL`	Board address that receives the veto notifications
`SENDER_ALIAS`	Station address used as sender
`MAILING_LIST_SHEET_ID`	The same Mailing List ID as in Module 1
`STATUTES_FILE_ID`	File ID of your statutes PDF
`PRIVACY_POLICY_FILE_ID`	File ID of your privacy policy PDF
`CREDITOR_ID`	Your SEPA creditor identifier (Gläubiger-ID)
`CREDITOR_ADDRESS`	Postal address of your association – printed on the SEPA mandate

Don't forget the HTML files: The forms contain the placeholder `[Your Station Name]` and the example address `hello@yourstation.com`. Use the editor's *Find and replace* function in each `Index` file to swap them for your own. The PDF template takes the name automatically from `STATION_NAME` and `LEGAL_NAME`.

12 Going Live, Authorization & Testing

12.1 Testing & Playing Around (The Sandbox)

Before sharing the live link with your community, you might want to test the form, change colors, or adjust the email text. Google Apps Script provides a built-in sandbox for this, so you don't accidentally break the live version while experimenting.

How to use the Test Deployment:

1. In the Apps Script Editor, click on **Deploy** and select **Test deployments**.
2. Under the *Web app* section, you will see a URL ending in `/dev` (unlike the live URL which ends in `/exec`).
3. Open this `/dev` link in a new browser tab. This is your personal testing ground. Only you (the developer logged into the Google account) can open this link.
4. You can now go back to the code, make changes (like rewriting the email template), and save the file.
5. Simply refresh the `/dev` tab in your browser to instantly see your changes!

12.2 Deploying the Web App (First Time Only)

Once you have pasted the code and entered your specific IDs, you need to publish the script so applicants can actually see the form on the internet.

1. In the Apps Script Editor, click on the blue **Deploy** button in the top right corner and select **New deployment**.
2. Click the gear icon next to "Select type" and choose **Web app**.
3. Under *Description*, type something like "Version 1".
4. Under *Execute as*, leave it as **Me** (your station's admin account).
5. **CRUCIAL:** Under *Who has access*, change the dropdown to **Anyone**. If you leave it restricted, DJs will get an error message when trying to apply!
6. Click **Deploy**. You will receive a long *Web App URL* (ending in `/exec`). Save this link! This is the URL you will later link or embed into your station's website.

12.3 Updating Later (Keep Your Links!)

Your live `/exec` link does **not** update automatically when you save the code. To publish changes:

1. Click **Deploy** > **Manage deployments**.
2. Select your existing web app deployment on the left and click the **pencil icon** (Edit).
3. Under *Version*, choose **New version** and add a short description.
4. Click **Deploy**. The URL shown afterwards is exactly the same as before – your website keeps working with the new code.

Never use "New deployment" for updates

"New deployment" creates a *second* URL. Your website would keep running the old code through the old link. If it happened by accident: edit the old deployment as described above and archive the new one.

12.4 The Scary (But Harmless) Google Security Warning

When you execute the script for the very first time (or authorize the deployment), Google will likely throw a highly intimidating red warning screen stating: **"Google hasn't verified this app."**

Do not panic! This happens simply because you are using a custom script that hasn't been officially submitted to Google's corporate review team (which costs money and takes weeks). Since you own the script and it runs securely within your own Google Workspace, it is completely safe to authorize it.

How to bypass it:

1. On the warning screen, look for the small text button that says **Advanced** and click it.
2. A new line of text will appear at the bottom. Click on **Go to [Your Project Name] (unsafe)**.
3. Click **Allow** to grant the script permission to read your calendar, edit your sheets, and send emails on your behalf.

This authorization usually only happens once. Afterwards, the tools will run smoothly in the background without ever bothering you or the applicants again.

12.5 The Executions Log (How to panic correctly)

When you click a button or submit a form and nothing happens, your first instinct might be to assume the code is permanently broken. However, Apps Script has a built-in diagnostic tool that tells you exactly what went wrong behind the scenes.

The Solution: Open the Apps Script Editor and click on the **Executions** tab (the list icon on the left sidebar). It logs every single time a script runs. If a script fails silently, it will show up here with a red "Failed" tag. Clicking on the failed run will highlight the exact error message and the specific line of code that caused the crash. This is invaluable information if you ever need to ask a developer or the community for help!

13 Data Privacy & Security (GDPR/DSGVO)

Because the station is handling highly sensitive financial and personal data, strict adherence to data protection principles is non-negotiable.

13.1 Security Built Into the Code

The web apps are public by design – anyone who knows the URL can open them. The code is therefore built so that this URL can't be misused:

- **Forgery-proof Accept buttons:** The links in the notification emails only contain a long random token. Email address, name and slot are always read from the spreadsheet, never from the link. Nobody can use the public URL to send emails in your station's name.
- **No double processing:** Accepting an application twice, or two people booking the same slot at the same moment, is not possible.
- **Clean input:** Everything applicants type in is checked by the script and made harmless before it appears in emails or pages. Entries starting with `=`, `+`, `-` or `@` are stored as plain text, so nobody can smuggle formulas into your spreadsheets.
- **Exact lookups:** The Schedule Checker only matches complete email addresses.

13.2 Securing the Master Spreadsheets

Your Google Sheets are no longer just lists of names; they are databases containing personal phone numbers, home addresses, and plain-text IBANs from your supporting members. You must strictly limit access. Do not share the spreadsheet links in open station group chats. Only the board of directors and the finance team should have "Editor" or "Viewer" access to the Supporting Members spreadsheet.

13.3 Privacy Information at the Point of Collection

The GDPR requires that people are informed about how their data is processed *when* it is collected. Link your privacy policy on the website page where the forms are embedded, ideally right above the iframe.

13.4 Routine Data Purges

To comply with data protection regulations, rejected show applications and vetoed membership requests should not be kept indefinitely. Make it a habit to delete rejected rows from the spreadsheets at the end of every calendar year. When someone unsubscribes from the newsletter, delete their row in the Mailing List.

13.5 Data Processing Agreement with Google

Since Google processes personal data on your behalf (form data, IBANs, email addresses), you should make sure a valid **Data Processing Agreement (DPA)** is in place with Google. For Google Workspace, this is usually automatically part of the terms of service – still, check in the Admin console whether it has been accepted.

13.6 Data Subject Rights (Access & Deletion)

If someone requests access to or deletion of their data, you'll need to handle this manually in the spreadsheets (Applications, SOS List, Members and Mailing List). The toolkit deliberately doesn't automate this process, since human review is appropriate here. Decide internally who on your team is responsible and how quickly you'll respond (legally, typically within one month).

Note: This chapter summarizes good practice; it is not legal advice.

14 Implement to your website (iFrame Integration)

Because we configured the Google Apps Script to allow external embedding (using the `ALLOWALL` X-Frame option in the code), you do not need to build complex API bridges or database connections to integrate these tools into your station's website. You can simply embed them using a standard HTML `iframe`.

This approach keeps the user seamlessly on your website, while Google's servers handle all the data processing and security in the background.

14.1 The Embedding Code

Once you have deployed your script as a Web App (ensuring access is set to "Anyone") and copied the `/exec` URL, you can pass it to your webmaster or drop it directly into your website builder (like WordPress, Webflow, or a custom frontend).

Use the following HTML snippet and replace the source link with your own:

```
<iframe
  src="YOUR_WEB_APP_URL_HERE"
  width="100%"
  height="850px"
  frameborder="0"
  scrolling="yes">
</iframe>
```

14.2 Tips for a Seamless Integration

- **Mobile Responsiveness:** Always set the width to `100%` rather than a fixed pixel amount. The CSS inside the forms is specifically designed to automatically scale down and look perfect on smartphone screens.
- **Scrolling:** Make sure `scrolling="yes"` is enabled. This is crucial for longer forms (like the show application or the membership form) so users can scroll through all the fields natively within the frame.
- **Height Adjustments:** Depending on your website's layout, you might need to adjust the `height="850px"` value. Set it long enough to display the majority of the form so the frame does not feel cramped, but not so long that it creates massive empty spaces on shorter pages (like the DJ Schedule Checker).
- **Updates:** Thanks to "Manage deployments > New version", the embedded link never has to change when you update the code.

15 License & Getting Help

This project is released under the **MIT License** – meaning any station is free to use, adapt, and redistribute the code. The full license text is available in the `LICENSE` file in the repository. If you get stuck during setup or find a bug, feel free to open an **Issue** on the GitHub repository, or email me at helenbolek@web.de.